

Exercise 9 Report

Student Name: Mohamad Aiman Danish Bin Mohamad Nawawi

Student ID: 52213124447

Lab Group: L02-B01

Date: August 25, 2026

Module A – NLP Pipeline Table

Step	What It Does	Example	Why It's Needed	Source
Tokenization	Splits raw text strings into smaller individual units called tokens (words, subwords, or symbols).	"I love AI" → ["I", "love", "AI"]	Converts raw text into discrete units necessary for standard vectorization and numerical feature engineering.	https://scikit-learn.org/stable/modules/feature_extraction.html
Stopword Removal	Removes high-frequency grammatical words that contain minimal sentiment or semantic value.	Removes "the", "is", "a"	Eliminates dataset noise and reduces vocabulary dimensionality so models focus on high-impact terms.	https://scikit-learn.org/stable/modules/feature_extraction.html
Stemming	Uses heuristic, rule-based chopping to reduce words to their raw crude root form.	"running" → "run"	Normalizes variant word forms into a single root to collapse sparse vocabulary sizes.	https://scikit-learn.org/stable/modules/feature_extraction.html
Lemmatization	Uses morphological analysis and context dictionaries to map words back to valid base lemmas.	"better" → "good"	Preserves true semantic meanings accurately without producing non-dictionary chopped words.	https://scikit-learn.org/stable/modules/feature_extraction.html

Module B - Feature Extraction Tables

Table B.1: Bag-of-Words vs. TF-IDF Comparison

Method	How It Works	Pros	Cons	Source
Bag-of-Words	Constructs a global vocabulary dictionary and counts raw term frequencies per document.	Simple to implement and computationally lightweight.	Discards word order, context, and heavily favors common non-informative words.	https://scikit-learn.org/stable/modules/feature_extraction.html
TF-IDF	Scales term frequency by multiplying it with the Inverse Document Frequency of each word across the corpus.	Automatically penalizes frequent non-informative words while highlighting rare, distinct terms.	Generates high-dimensional sparse matrices and fails to capture deep semantic dependencies.	https://scikit-learn.org/stable/modules/feature_extraction.html

Table B.2: Advanced Encoding Techniques

Encoding	Captures Semantic s?	Dimension ality	Pre- traine d?	Source
Bag-of- Words	No	High (Vocabulary Size)	No	https://scikit-learn.org/stable/modules/feature_extraction.html
TF-IDF	Partially	High (Vocabulary Size)	No	https://scikit-learn.org/stable/modules/feature_extraction.html
Word2Vec	Yes	Low (100–300 dimensions)	Yes	https://code.google.com/archive/p/word2vec/
GloVe	Yes	Low (50–300 dimensions)	Yes	https://nlp.stanford.edu/projects/glove/
BERT Embeddin gs	Yes	Low (768–1024 dimensions)	Yes	https://arxiv.org/abs/1810.04805

Module C - Classifier Performance Comparison

Classifier	Accuracy	Precision	Recall	F1-Score	Source
Naive Bayes (Multinomial)	1.0000	1.0000	1.0000	1.0000	https://scikit-learn.org/stable/modules/naive_bayes.html
SVM (Linear Kernel)	1.0000	1.0000	1.0000	1.0000	https://scikit-learn.org/stable/modules/svm.html

Module D - New Text Inference Predictions

Input Review Text	Predicted Sentiment	Correct Classification?	Source
"This restaurant is amazing, I love the food!"	Positive	Yes	VS Code Terminal Output
"Terrible service, will never come back."	Negative	Yes	VS Code Terminal Output
"The food was okay, nothing special."	Positive	Misclassified (Neutral/Moderate text)	VS Code Terminal Output
"Best pizza I have ever had!"	Negative	Misclassified (Vocabulary mismatch)	VS Code Terminal Output
"Worst experience of my life."	Negative	Yes	VS Code Terminal Output

```
Problems  Output  Debug Console  Terminal
PS C:\Users\Aiman Danish\OneDrive\Desktop\Exercise9_MohamadAimanDanishBinMohamadNawawi_52213124447_L02B01> python nlp_classification.py
--- MODEL PERFORMANCE ---
Naive Bayes Accuracy: 1.0000
SVM Accuracy: 1.0000

--- NEW PREDICTIONS ---
Review: 'This restaurant is amazing, I love the food!' --> Positive
Review: 'Terrible service, will never come back.' --> Negative
Review: 'The food was okay, nothing special.' --> Positive
Review: 'Best pizza I have ever had!' --> Negative
Review: 'Worst experience of my life.' --> Negative
PS C:\Users\Aiman Danish\OneDrive\Desktop\Exercise9_MohamadAimanDanishBinMohamadNawawi_52213124447_L02B01>
```

Figure 1 Terminal Execution Output of Model Evaluation and Inference

Core NLP Challenges & Model Performance Analysis

1. Sarcasm and Complex Sentiment Modifiers

Bag-of-Words and TF-IDF representations fail when processing sarcastic phrases like *"Oh, fantastic, another delayed train!"* because frequency-based vectorizers assign strong positive weights to words like "fantastic", completely missing the negative underlying tone.

2. Out-of-Vocabulary (OOV) Terms and Short Synthetic Datasets

In small datasets, models struggle with unseen vocabulary during inference. For instance, in our test prediction, "Best pizza I have ever had!" was classified as **Negative**. Because "pizza" and "best" were absent from the limited training corpus vocabulary, the linear model defaulted to negative weights based on out-of-vocabulary fallback artifacts.

3. Polysemy and Contextual Ambiguity

Traditional static vector space representations cannot handle polysemy—where a single word changes meaning based on surrounding context. Words like "bank" (financial institution vs. river edge) receive fixed feature counts in TF-IDF, requiring contextualized transformer architectures like BERT to resolve semantic context dynamically.